



A CSR initiative by



kotak
Kotak Mahindra Bank

Kotak IISc AI-ML Centre



TANUH
Translational Artificial Intelligence
for Networked Universal Healthcare

Topics in Large Language Models

Lecture Series - Part I

Sriram Ganapathy

Electrical Engineering, IISc.

Contents - This Lecture

- ✓ 1.1 LLMs - Recap
- ✓ 1.2 In-context learning, low-rank adaptation
- ✓ 1.3 Retrieval Augmented Generation.
 - *1.3.1 Principle
 - *1.3.2 Recent advances

Attention is all you need

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Transformers

- ✓ Key building blocks

- * Attention, Residual connection, normalization, feed-forward layers and positional encoding

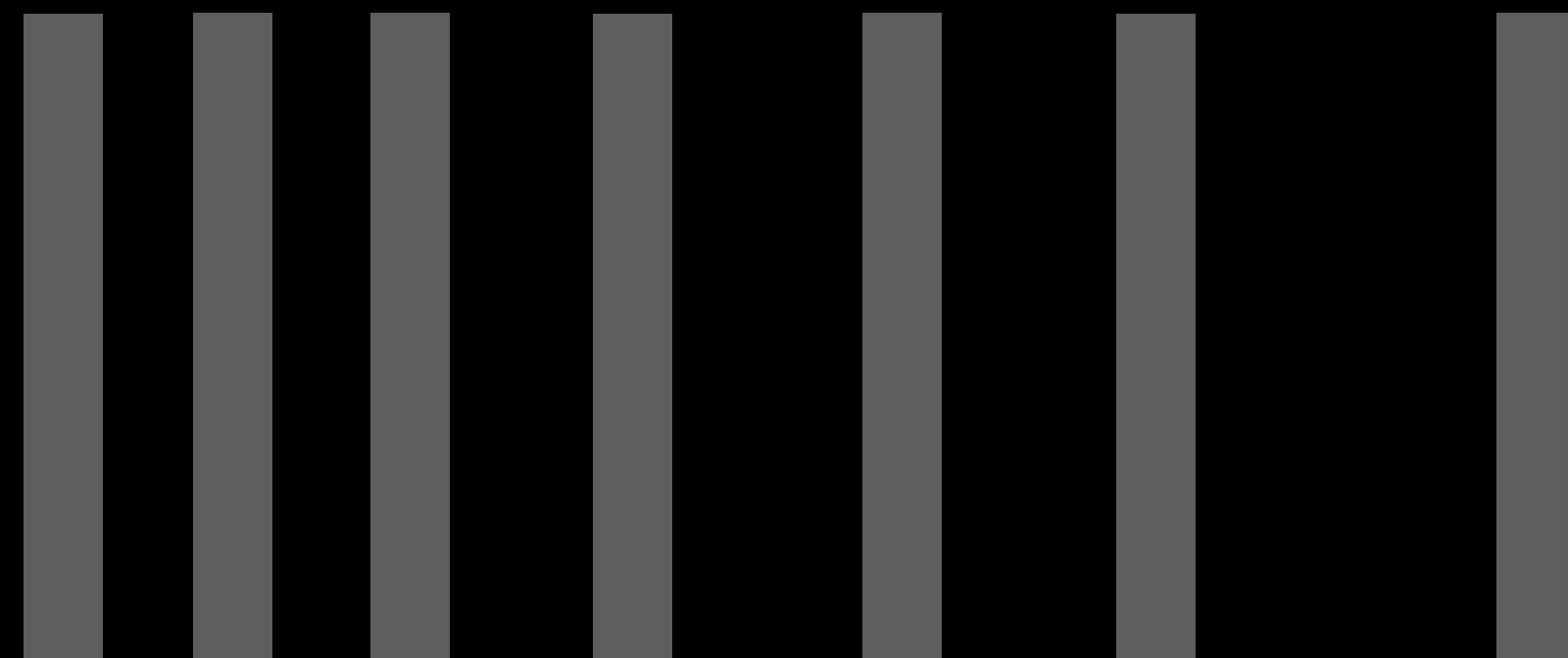
- ✓ Big picture

- * Every input representation “attends” to every other input representation including itself.
 - * Generated a weighted combination of all representations, where the weights are based on the similarity between representations.

Self-attention

- ✓ Assume that words are represented as vectors in some embedding space (for example, word2vec). Let $\mathbf{x}_t \in \mathcal{R}^{1 \times D}$

Word
Embeddings



I am a student of machine learning

$\mathbf{x}_1$

$\mathbf{x}_2$

$\mathbf{x}_3$

$\mathbf{x}_4$

$\mathbf{x}_5$

$\mathbf{x}_6$

$\mathbf{x}_7$

Self-attention

✓ There is nothing learnable in using input directly —> Use transformed versions of the input.

$\mathbf{k}_1$	$\mathbf{q}_1 \mathbf{k}_1^T$	$\mathbf{q}_1 \mathbf{k}_2^T$					
$\mathbf{k}_2$							
$\mathbf{k}_3$							
	$\mathbf{q}_1$	$\mathbf{q}_2$	$\mathbf{q}_3$				

With

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_T \end{bmatrix} \in \mathcal{R}^{T \times D}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_{\mathbf{Q}} \in \mathcal{R}^{T \times d}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_{\mathbf{K}} \in \mathcal{R}^{T \times d}$$

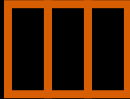
$$\mathbf{V} = \mathbf{X}\mathbf{W}_{\mathbf{V}} \in \mathcal{R}^{T \times d}$$

$$\mathbf{A} = \mathbf{Q}\mathbf{K}^T \in \mathcal{R}^{T \times T}$$

$$\mathbf{E} = \text{softmax}(\frac{\mathbf{A}}{d})V \in \mathcal{R}^{T \times d}$$

Scaled dot-product attention

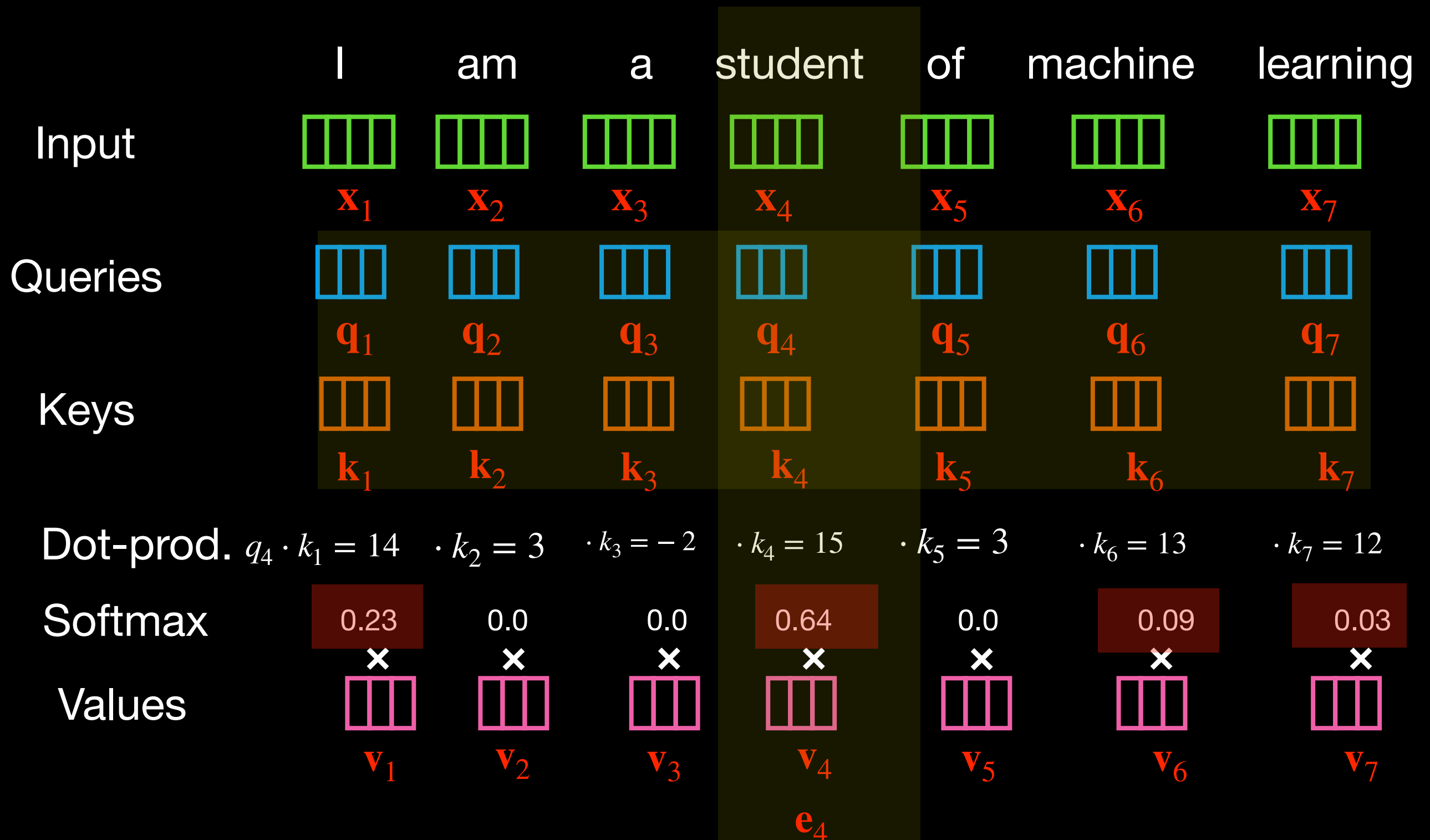
Closer look at the operations

	I	am	a	student	of	machine	learning
Input	 $\mathbf{x}_1$	 $\mathbf{x}_2$	 $\mathbf{x}_3$	 $\mathbf{x}_4$	 $\mathbf{x}_5$	 $\mathbf{x}_6$	 $\mathbf{x}_7$
Queries	 $\mathbf{q}_1$	 $\mathbf{q}_2$	 $\mathbf{q}_3$	 $\mathbf{q}_4$	 $\mathbf{q}_5$	 $\mathbf{q}_6$	 $\mathbf{q}_7$
Keys	 $\mathbf{k}_1$	 $\mathbf{k}_2$	 $\mathbf{k}_3$	 $\mathbf{k}_4$	 $\mathbf{k}_5$	 $\mathbf{k}_6$	 $\mathbf{k}_7$

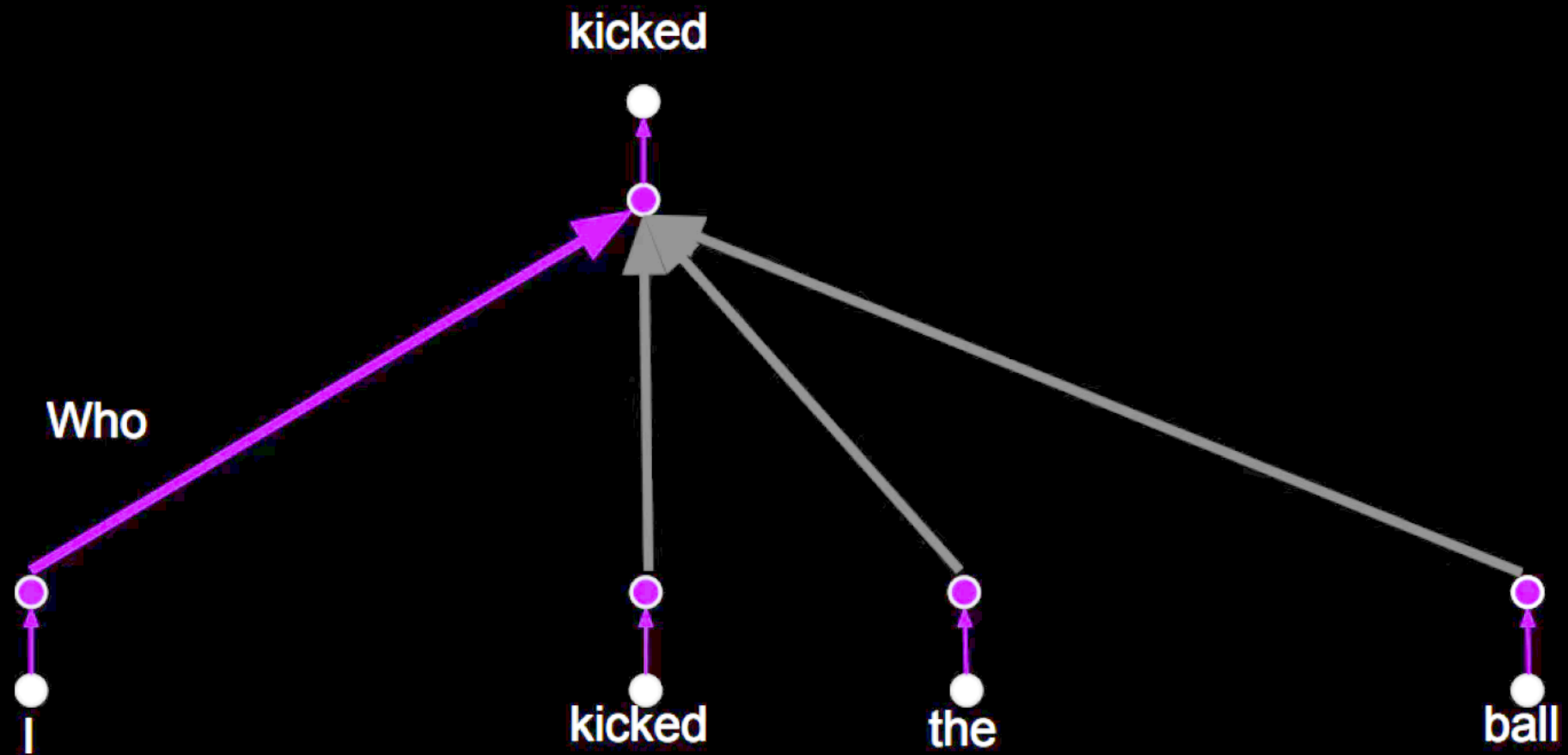
Closer look at the operations

	I	am	a	student	of	machine	learning
Input							
	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_5$	$\mathbf{x}_6$	$\mathbf{x}_7$
Queries							
	$\mathbf{q}_1$	$\mathbf{q}_2$	$\mathbf{q}_3$	$\mathbf{q}_4$	$\mathbf{q}_5$	$\mathbf{q}_6$	$\mathbf{q}_7$
Keys							
	$\mathbf{k}_1$	$\mathbf{k}_2$	$\mathbf{k}_3$	$\mathbf{k}_4$	$\mathbf{k}_5$	$\mathbf{k}_6$	$\mathbf{k}_7$
Dot-prod.	$q_4 \cdot k_1 = 14$	$\cdot k_2 = 3$	$\cdot k_3 = -2$	$\cdot k_4 = 15$	$\cdot k_5 = 3$	$\cdot k_6 = 13$	$\cdot k_7 = 12$
Softmax	0.23	0.0	0.0	0.64	0.0	0.09	0.03

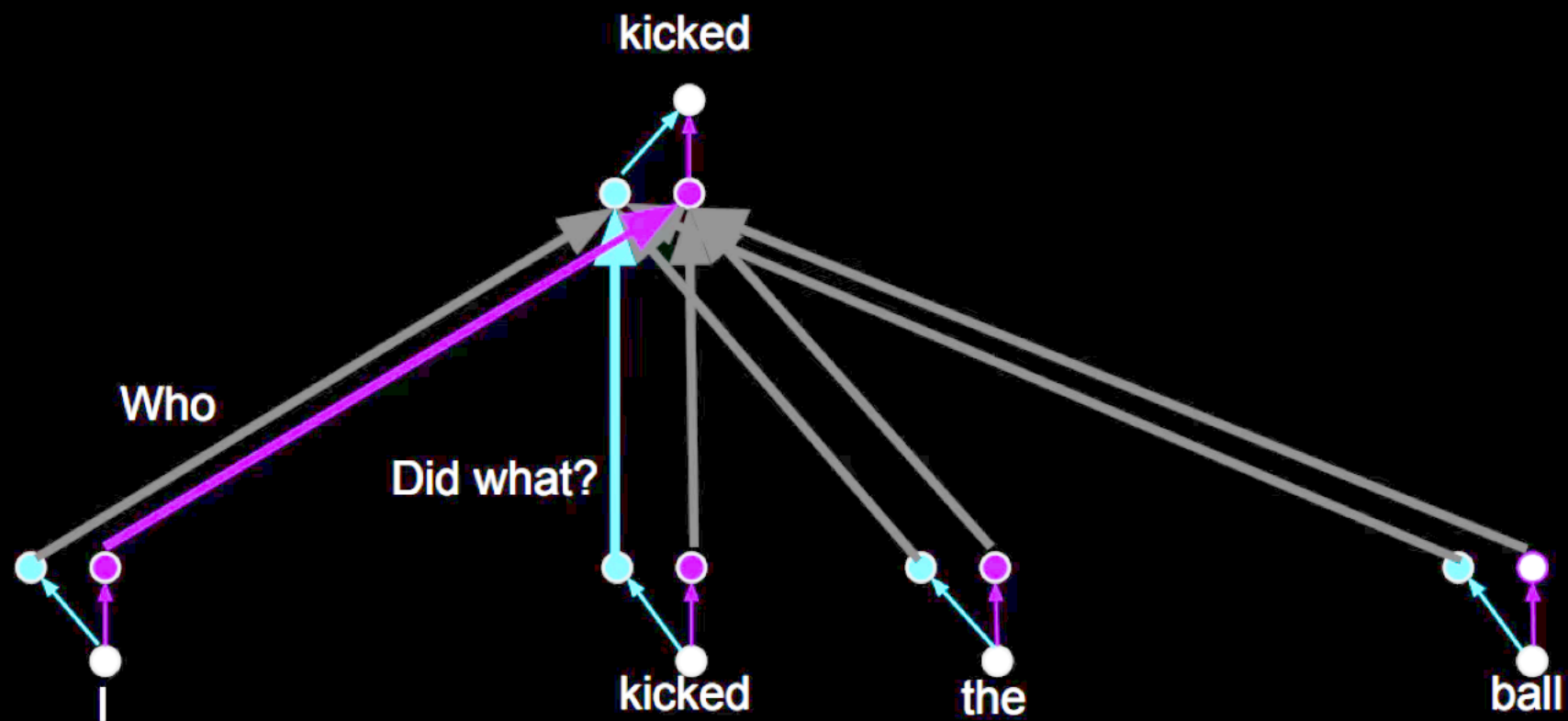
Closer look at the operations



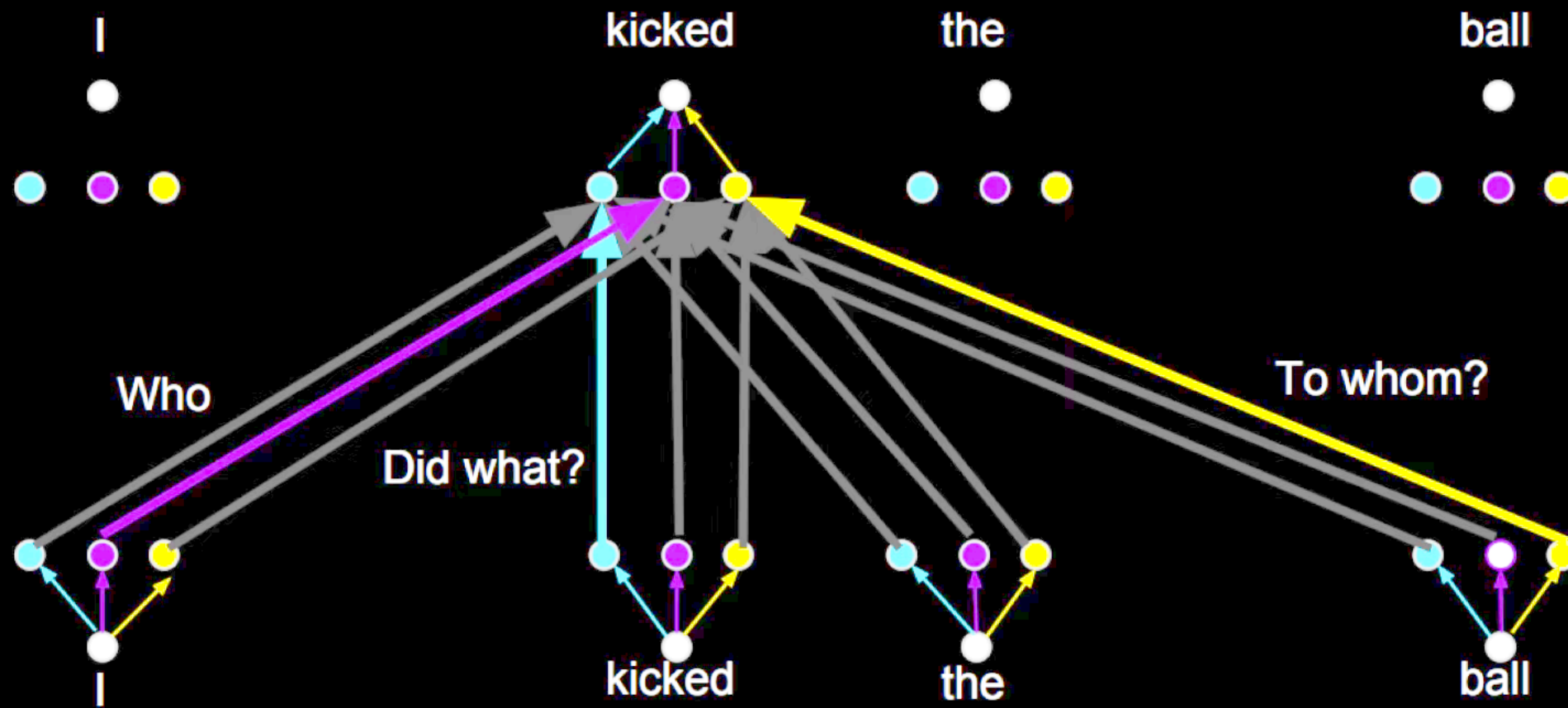
Multi-head attention



Multi-head attention



Multi-head attention



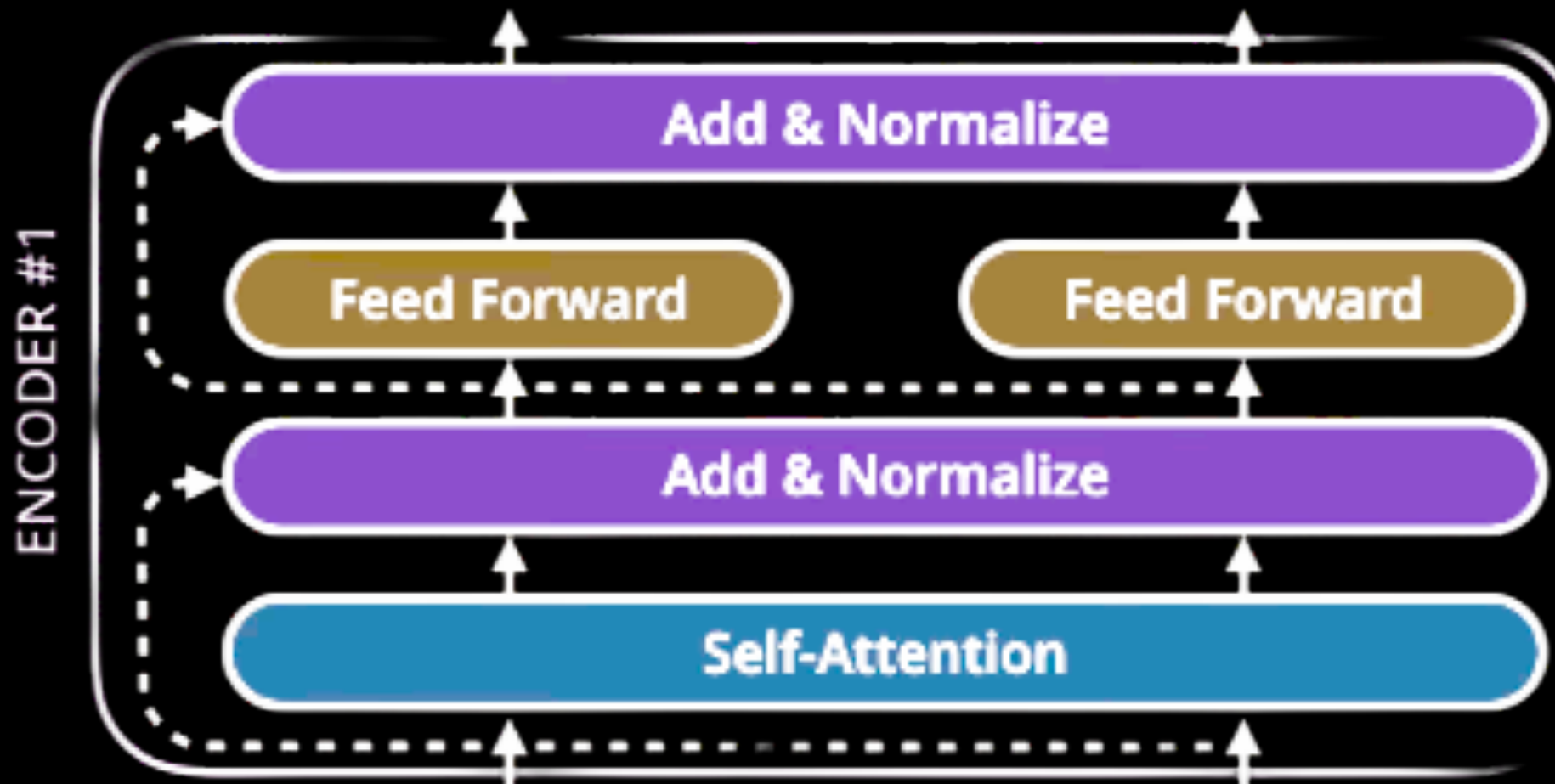
Multi-head attention

- ✓ Concatenate attention from multiple heads,

$$\mathbf{E}_k = \text{softmax}\left(\frac{\mathbf{A}_k}{d_k}\right)V_k \in \mathcal{R}^{T \times d_k} \quad \mathbf{E} = [\mathbf{E}_1 \dots \mathbf{E}_h]$$

- ✓ Typically, $d = h \times d_k$
- ✓ The output of Multi-head attention goes to subsequent processing steps

Transformer single processing layer

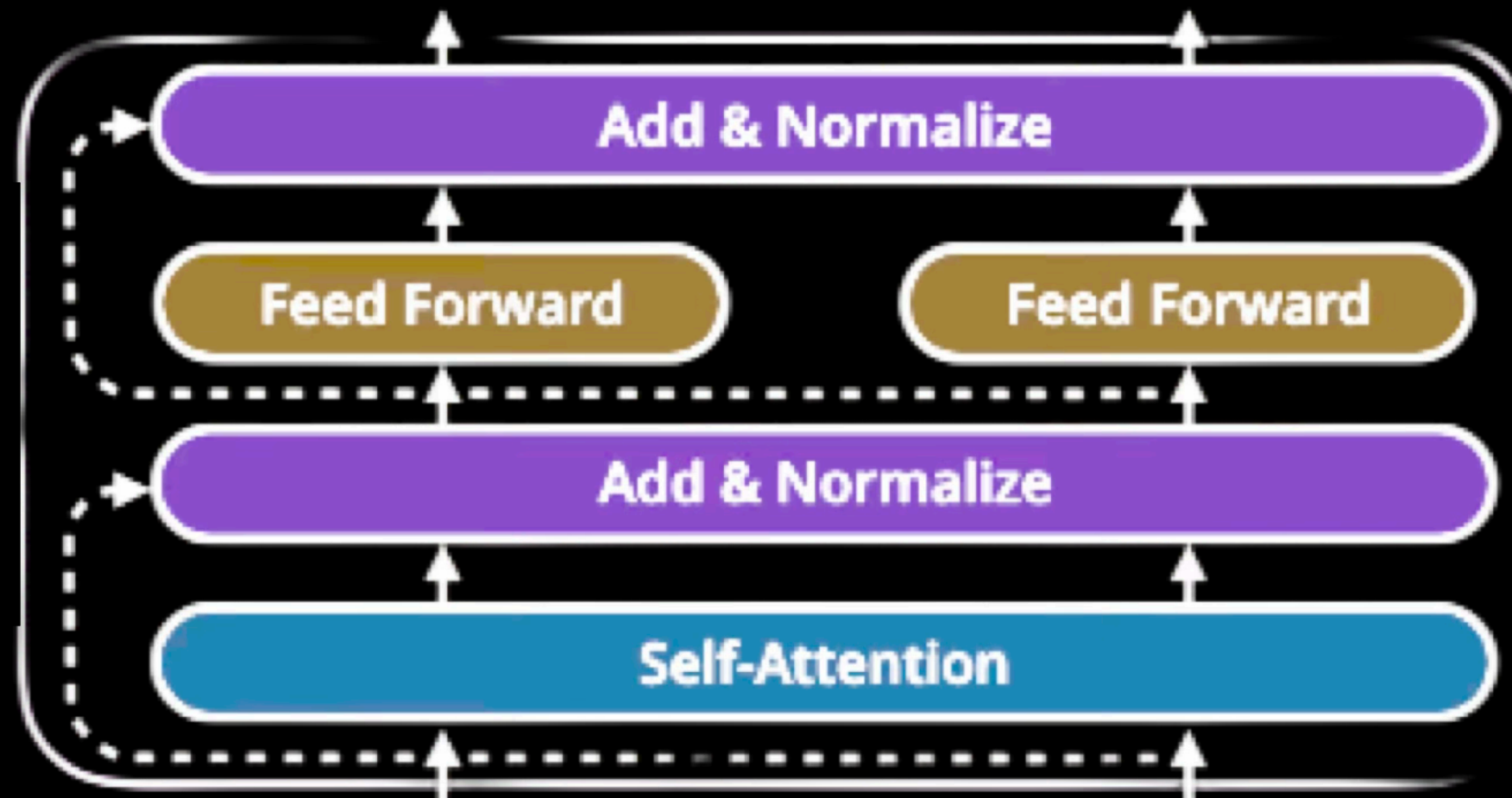


Layer norm - A normalization for each vector, rescales and recenters the activations **across the feature dimension(s)**.

Residual connections - Increases the gradient flow and stability.

Decoder only models

- ✓ Model contains only self-attention layers



- ✓ Difference between encoder only models and decoder only models is in the loss function

Decoder only models

- ✓ Model contains only self-attention layers
- ✓ Difference between encoder only models and decoder only models is in the loss function
- * Loss function in a decoder-only model $\mathcal{L} = - \sum_t P(\mathbf{x}_t | \mathbf{x}_{<t})$
- * Model is learned to predict the next output prediction in a sequence
 - ◎ Most of the language modeling and creative generation tasks can be represented as a slot-filling task
 - For example, question answering can be considered as structured next token prediction of the form (<Question>, ...)

Evolution of Transformer models

✓ Recent efforts

* Decoder only models

- ◎ Uses causal attention in an auto-regressive setting
- ◎ Trained with next token prediction losses in a self-supervised fashion.
- ◎ Useful as a general purpose model for wide-range of tasks.
 - Allows free form generation
- ◎ Popular examples - most of the LLMs - GPT series [Radford 2018, ...], Gemini [Team 2023, ...], LLaMa [Touvron 2023, ...], etc

Large Language Models

- ✓ Extending the task of self-supervision
- ✓ Mine lots of text, audio and visual data (and others)
 - * Crawled from the web, as well as, from other resources.
- ✓ Design the **model with large capacity** (Millions —> Billions of parameters)
- ✓ Pre-train the model
 - * With MLM, next-token prediction and similar style of losses
 - * High resource of computations.
- ✓ Final trained model can be **fine-tuned for supervised tasks**
 - * Load the parameters as initialization and perform supervised learning.

Time for some ...

Q & A

Contents - This Lecture

- ✓ 1.1 LLMs - Recap
- ✓ 1.2 In-context learning, low-rank adaptation
- ✓ 1.3 Retrieval Augmented Generation.
 - *1.3.1 Principle
 - *1.3.2 Recent advances

Contents - This Lecture

- ✓ 1.1 LLMs - Recap
- ✓ 1.2 In-context learning, low-rank adaptation
- ✓ 1.3 Retrieval Augmented Generation.
 - *1.3.1 Principle
 - *1.3.2 Recent advances

1.1 Deep Foundation Models

- ✓ Moving trained models for tasks and domains beyond ones seen in training
 - * Model parameters frozen —> In-context learning
 - * Model adaptation with only a small number of parameters —> Low-rank adaptation

In-context learning

Language Models are Few-Shot Learners

Tom B. Brown*

Benjamin Mann*

Nick Ryder*

Melanie Subbiah*

Jared Kaplan[†]

Prafulla Dhariwal

Arvind Neelakantan

Pranav Shyam

Girish Sastry

Amanda Askell

Sandhini Agarwal

Ariel Herbert-Voss

Gretchen Krueger

Tom Henighan

Rewon Child

Aditya Ramesh

Daniel M. Ziegler

Jeffrey Wu

Clemens Winter

Christopher Hesse

Mark Chen

Eric Sigler

Mateusz Litwin

Scott Gray

Benjamin Chess

Jack Clark

Christopher Berner

Sam McCandlish

Alec Radford

Ilya Sutskever

Dario Amodei

OpenAI

In-context learning

✓ Decoder only models can

* Learn a new-task purely using examples

* For example

◎ (Federer, Tennis), (Kohli, ____) ==>

- Model learns the relationship between pairs given and answers "Cricket"

◎ (Federer, Switzerland), (Ronaldo, ____) ==>

- Now the model answers as "Portugal"

In-context learning

✓ Formally

* The input is of the form

⊙ $(\mathcal{X}_1, \mathcal{Y}_1), (\mathcal{X}_2, \mathcal{Y}_2), \dots, (\mathcal{X}_n, \mathcal{Y}_n), (\mathcal{X}_{n+1}, \dots)$

⊙ The model learns the mapping function $f(\mathcal{X}) \rightarrow \mathcal{Y}$

- The weights of the model are frozen

⊙ How is it possible

- Each transformer layer builds an internal representation of the examples in the context.
- The self-attention mechanism allows the model to dynamically “retrieve” relevant examples when processing a new query token.
- This leads to behavior resembling **few-shot learning** — i.e., learning from a few examples

In-context learning

✓ Types of In-context learning

Type	Description	Example
Zero-shot ICL	The model performs a task using only a natural-language instruction. (Model has not seen this task during fine-tuning)	"Translate to French: apple → ?"
One-shot ICL	The model is given one example of input-output.	"apple → pomme, banana → ?"
Few-shot ICL	The model is given multiple examples to infer a pattern.	cat → chat dog → chien house → maison tree → ?

Why does in-context learning work

Published as a conference paper at ICLR 2023

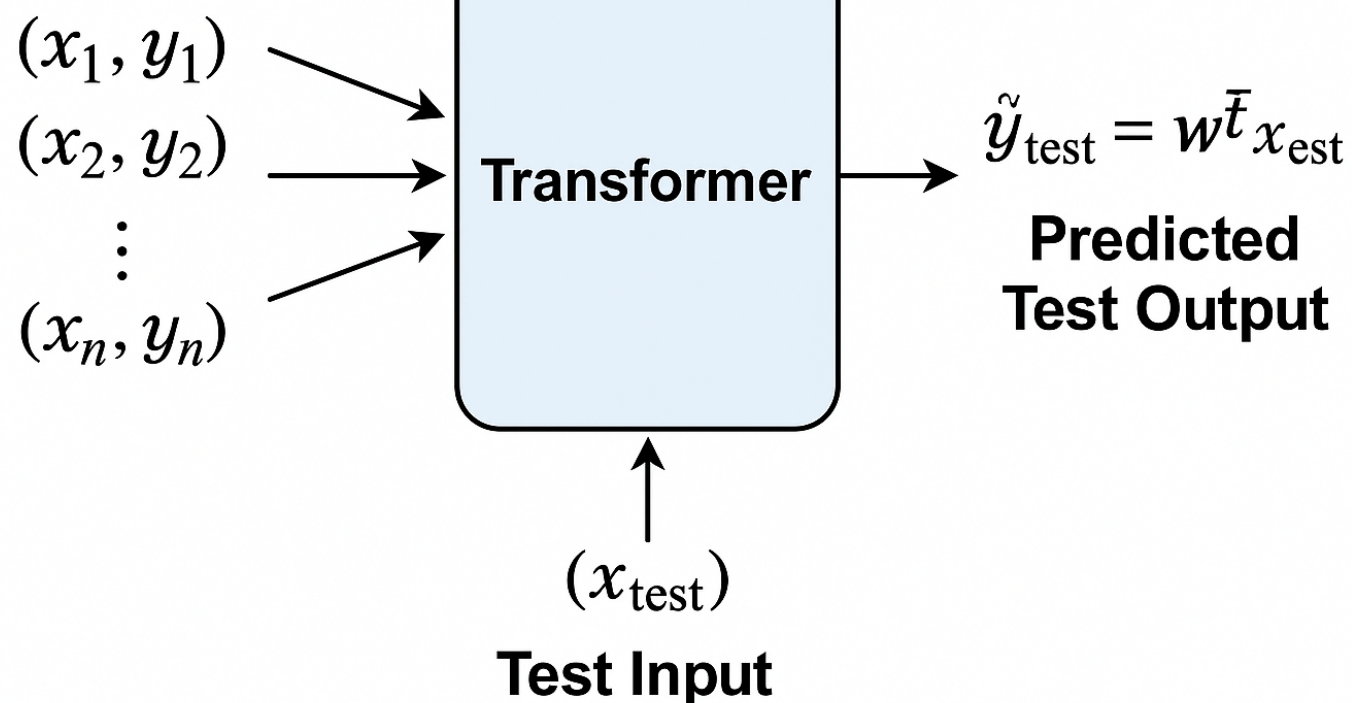
WHAT LEARNING ALGORITHM IS IN-CONTEXT LEARNING? INVESTIGATIONS WITH LINEAR MODELS

Ekin Akyürek^{1,2,a} Dale Schuurmans¹ Jacob Andreas^{*2} Tengyu Ma^{*1,3,b} Denny Zhou^{*1}

¹Google Research ²MIT CSAIL ³Stanford University ^{*}collaborative advising

Regression-based In-Context Learning

Examples



Linear
Regression

Linear Regression

$$(X^T X + \lambda I)^{-1} X^T y$$

y_{test}

The variables needed to compute linear regression outputs can be stored in the hidden activations during the forward pass through the input itself

Low-rank adaptation

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**

Microsoft Corporation

{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com

yuanzhil@andrew.cmu.edu

(Version 2)

Low-rank adaptation

- ✓ Given a pre-trained transformer model with weights (for example, let $W_0 \in \mathcal{R}^{D \times d}$)
- * Low-rank adaptation freezes the W_0
 - ⊙ But learns a smaller number of parameters
 - $\Delta W = BA$
 - $W = W_0 + \alpha \cdot \Delta W = W_0 + \alpha \cdot BA,$
 - Where α is a scaling factor. $A \in \mathcal{R}^{r \times d}, B \in \mathcal{R}^{D \times r}$
 - Only the matrices A, B are learned from the data

Low-rank adaptation

✓ Parameter saving by low-rank adaptation

* If one were to update the base model W_0

◎ Need to learn $D \times d$ parameters

* Learning only A, B parameters

- Need to learn $r(D + d)$ parameters

- Savings for $D = 1024, d = 128, r = 8$ is

- $8(1024 + 128)/(1024 \times 128) = 0.0703$ (7% parameters are alone updated)

◎ Methods like LORA fall into larger category of methods called parameter efficient fine-tuning (PEFT).

Low-rank adaptation

- ✓ LoRA is typically inserted into linear projection layers of transformer blocks:
 - * Query (Q) and Value (V) projection matrices in self-attention layers.
 - * Rarely also in feedforward networks.
- ✓ Why is it very successful — because downstream tasks often require **specific but limited shifts** in representation space, not a complete re-learning of all features.

Time for some ...

Q & A



Thank You